

TRƯỜNG ĐẠI HỌC KHOA HỌC HUẾ



ĐỀ TÀI: HỆ THỐNG KHÓA CỬA THÔNG MINH

DỰA TRÊN ESP32 VÀ RFID

(ESP32 + RFID RC522 + Servo SG90)

Giảng viên hướng dẫn:	Võ Việt Dũng
Nhóm học phần:	Phát triển ứng dụng IoT - Nhóm 4
Mã lớp học phần:	2025-2026.2.TIN4024.004
Sinh viên thực hiện:	Nguyễn Thành Doanh
Mã sinh viên:	22T1020071

Huế, Tháng 04 Năm 2026

MỤC LỤC

LỜI CẢM ƠN

CHƯƠNG 1: GIỚI THIỆU

- 1.1 Lý do thực hiện
- 1.2 Mục tiêu và yêu cầu
 - 1.2.1 Mục tiêu
 - 1.2.2 Yêu cầu
- 1.3 Phạm vi đề tài

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

- 2.1 Vi điều khiển ESP32
- 2.2 Module RFID RC522
- 2.3 Servo SG90
- 2.4 Giao thức Wi-Fi và HTTP
- 2.5 Telegram BotAPI

CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG

- 3.1 Giới thiệu các linh kiện
- 3.2 Sơ đồ kết nối và nguyên lý hoạt động
 - 3.2.1 Sơ đồ kết nối
 - 3.2.2 Nguyên lý hoạt động

CHƯƠNG 4: THIẾT KẾ PHẦN MỀM VÀ TRIỂN KHAI

- 4.1 Nền tảng và công cụ sử dụng
 - 4.1.1 Môi trường mô phỏng phần cứng Wokwi
 - 4.1.2 Nền tảng điều khiển Telegram Bot API
 - 4.1.3 Ngôn ngữ lập trình và Thư viện hỗ trợ
- 4.2 Giải thích code chính (main.cpp)
- 4.3 Tích hợp Telegram Bot
 - 4.3.1 Tạo Telegram Bot và cấu hình
 - 4.3.2 Điều khiển 2 chiều qua Inline Keyboard

CHƯƠNG 5: KẾT QUẢ THỬ NGHIỆM

- 5.1 Kết quả mô phỏng
- 5.2 Hạn chế và hướng phát triển

CHƯƠNG 6: KẾT LUẬN

TÀI LIỆU THAM KHẢO

LỜI CẢM ƠN

Trong quá trình thực hiện đề tài này, em đã nhận được nhiều sự hỗ trợ quý báu từ thầy cô và bạn bè. Em xin gửi lời cảm ơn chân thành đến:

Thầy Võ Việt Dũng đã tận tình hướng dẫn, góp ý và đồng hành cùng em trong suốt quá trình thực hiện đề tài. Những kiến thức chuyên môn và định hướng của thầy là nền tảng quan trọng giúp em hoàn thành bài báo cáo.

Em cũng xin cảm ơn quý thầy cô trong Khoa Công nghệ Thông tin đã truyền đạt những kiến thức cần thiết, tạo điều kiện thuận lợi để em có thể vận dụng vào thực tế, đặc biệt trong lĩnh vực IoT và lập trình nhúng.

Mặc dù đã cố gắng hoàn thiện, bài báo cáo vẫn khó tránh khỏi những thiếu sót. Em rất mong nhận được những ý kiến đóng góp từ thầy cô để có thể cải thiện và hoàn thiện hơn trong các đề tài sau.

Em xin chân thành cảm ơn!

CHƯƠNG 1: GIỚI THIỆU

1.1 Lý do thực hiện

Trong bối cảnh phát triển mạnh mẽ của công nghệ Internet of Things (IoT), nhu cầu bảo mật thông minh cho nhà ở và các công trình ngày càng trở nên cấp thiết. Hệ thống khóa cửa truyền thống sử dụng chìa khóa cơ học tồn tại nhiều hạn chế như dễ mất, dễ làm giả và không thể giám sát từ xa. Sự ra đời của các công nghệ RFID (Radio Frequency Identification) và vi điều khiển tích hợp Wi-Fi như ESP32 đã mở ra cơ hội xây dựng hệ thống kiểm soát ra vào thông minh, an toàn và tiện lợi hơn rất nhiều. Chính vì vậy, đề tài 'Hệ thống khóa cửa thông minh dựa trên ESP32 và RFID' là một giải pháp phù hợp và cần thiết trong thực tế.

1.2 Mục tiêu và yêu cầu

1.2.1 Mục tiêu

- Nhận dạng thẻ RFID và phân quyền mở/từ chối truy cập an toàn.
- Điều khiển servo motor SG90 để mở/đóng khóa cửa vật lý một cách tự động.
- Hiển thị trạng thái hệ thống theo thời gian thực trên màn hình LCD 16x2.
- Gửi thông báo tức thời qua Telegram Bot khi có sự kiện xảy ra.
- Cảnh báo bằng LED và buzzer khi có truy cập bất hợp lệ.
- Cho phép điều khiển 2 chiều qua Inline Keyboard Button của Telegram.
- Kết nối Wi-Fi để tích hợp dịch vụ đám mây và quản lý từ xa.

1.2.2 Yêu cầu

- Độ chính xác của RFID: Module RC522 phải đọc UID thẻ chính xác, phân biệt thẻ hợp lệ và không hợp lệ.
- Phản hồi nhanh: Servo và LED phản hồi trong vòng 100ms sau khi xác thực.
- Hoạt động ổn định: Hệ thống phải vận hành liên tục, không bị treo hay mất kết nối Telegram.
- Bảo mật: Danh sách UID hợp lệ được quản lý chặt chẽ, chỉ admin mới có thể thêm/xóa thẻ.
- Dễ mở rộng: Code cấu trúc rõ ràng, dễ thêm tính năng mới như nhận diện vân tay, lưu lịch sử.

1.3 Phạm vi đề tài

Đề tài tập trung vào việc xây dựng và mô phỏng hệ thống khóa cửa thông minh trên nền tảng Wokwi Simulator kết hợp với PlatformIO và VS Code. Toàn bộ phần cứng gồm ESP32, RFID

RC522, Servo SG90, LCD 16x2, LED và Buzzer đều được mô phỏng trong môi trường ảo. Hệ thống tích hợp Telegram Bot để giám sát và điều khiển từ xa qua Internet, phù hợp cho các ứng dụng bảo mật nhà ở quy mô nhỏ đến vừa.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Vi điều khiển ESP32

ESP32 là vi điều khiển 32-bit lõi kép do Espressif Systems sản xuất, sử dụng bộ vi xử lý Xtensa LX6 hoạt động ở tần số tối đa 240 MHz. Điểm nổi bật của ESP32 là tích hợp sẵn Wi-Fi 802.11 b/g/n và Bluetooth 4.2/BLE, giúp kết nối Internet mà không cần module bổ sung. Với RAM 520 KB và Flash 4 MB, ESP32 có thể xử lý đồng thời nhiều tác vụ phức tạp, từ đọc RFID, điều khiển servo, đến giao tiếp HTTPS với Telegram.

Thông số	Giá trị
CPU	Xtensa LX6 lõi kép, 240 MHz
Flash	4 MB
RAM	520 KB SRAM
Wi-Fi	802.11 b/g/n (2.4 GHz)
GPIO	34 chân (PWM, ADC, DAC, I2C, SPI, UART)
Giao thức	SPI, I2C, UART, I2S, CAN

Bảng 1: Thông số kỹ thuật ESP32 DevKit V1

2.2 Module RFID RC522

Module RFID MFRC522 hoạt động ở tần số 13.56 MHz, sử dụng giao thức SPI để giao tiếp với vi điều khiển. Mỗi thẻ RFID có một UID (Unique Identifier) từ 4 đến 7 byte duy nhất, được dùng làm mã xác thực trong hệ thống. Module có thể đọc thẻ Mifare Classic, Mifare Ultralight và các thẻ tương thích ISO/IEC 14443A.

Chân RFID	Chân ESP32	Chức năng
SDA (SS)	GPIO 5	Chọn thiết bị SPI
SCK MOSI	GPIO 18	Xung đồng hồ SPI
MISO RST	GPIO 23	Dữ liệu chủ → tớ
GND 3.3V	GPIO 19	Dữ liệu tớ → chủ
	GPIO 4 GND	Reset module Đất
	3V3	chung Nguồn cấp

Bảng 2: Kết nối chân RFID RC522 với ESP32

2.3 Servo SG90

Servo SG90 là servo nhỏ gọn, điều khiển bằng tín hiệu PWM với chu kỳ 20ms. Góc quay từ 0° đến 180°. Trong đề tài, servo đóng vai trò cơ cấu khóa vật lý: 0° là trạng thái đóng cửa, 90° là trạng thái mở cửa. Servo nhận tín hiệu PWM từ GPIO 13 của ESP32 thông qua thư viện ESP32Servo.

2.4 Giao thức Wi-Fi và HTTP

ESP32 sử dụng Wi-Fi 802.11 b/g/n để kết nối mạng và Internet. Hệ thống thực hiện HTTP POST để gửi thông báo lên Telegram và HTTP GET (polling) mỗi 2 giây để nhận lệnh điều khiển từ người dùng. Tất cả kết nối đều sử dụng HTTPS được mã hóa SSL/TLS thông qua thư viện WiFiClientSecure.

2.5 Telegram Bot API

Telegram Bot API cho phép gửi và nhận tin nhắn qua HTTPS. Bot được tạo thông qua @BotFather và nhận BOT_TOKEN xác thực. Hệ thống sử dụng endpoint sendMessage để gửi thông báo và getUpdates để nhận lệnh. Đặc biệt, tính năng Inline Keyboard cho phép nhúng nút bấm trực tiếp vào tin nhắn, giúp người dùng điều khiển hệ thống mà không cần gõ lệnh thủ công.

CHƯƠNG 3: KIẾN TRÚC HỆ THỐNG

3.1 Giới thiệu các linh kiện

Hệ thống được tổ chức theo mô hình IoT 3 lớp: lớp cảm biến/chấp hành, lớp xử lý trung tâm và lớp ứng dụng/giám sát.

Lớp	Thành phần	Chức năng
Cảm biến / Chấp hành	RFID RC522, Servo SG90, LED, Buzzer, LCD 16x2	Thu thập dữ liệu, thực thi lệnh vật lý
Xử lý trung tâm	ESP32 DevKit V1	Xử lý logic, xác thực, kết nối Wi-Fi
Ứng dụng / Giám sát	Telegram Bot + Inline Keyboard	Thông báo real-time, điều khiển từ xa

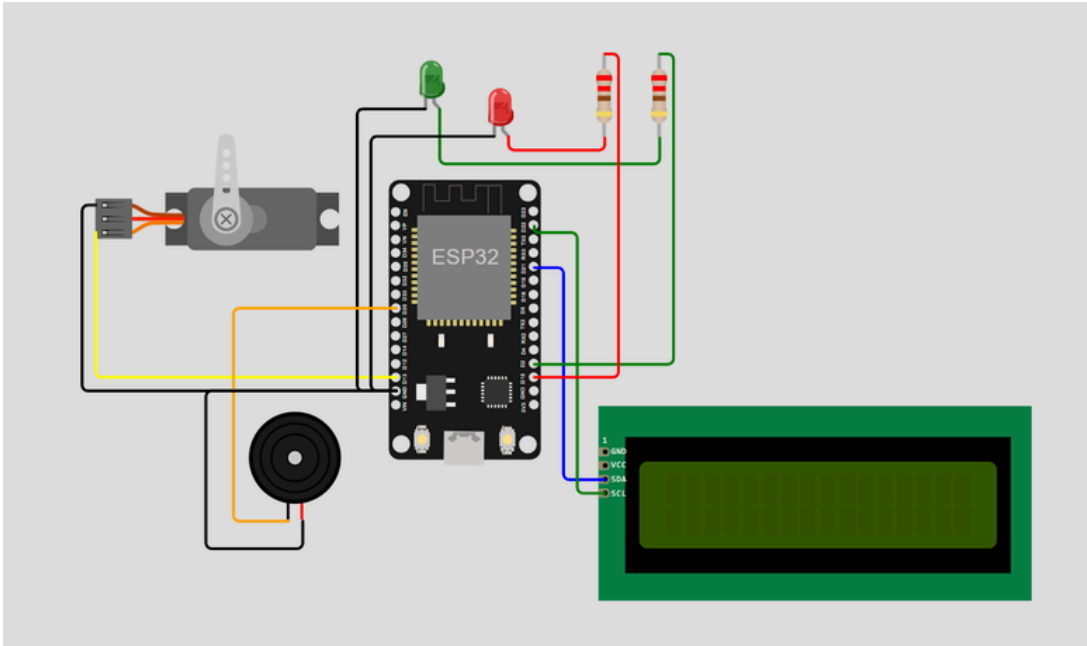
Bảng 3: Kiến trúc phân tầng hệ thống IoT

3.2 Sơ đồ kết nối và nguyên lý hoạt động

3.2.1 Sơ đồ kết nối

Toàn bộ kết nối phần cứng được mô phỏng trên Wokwi qua file diagram.json. Các linh kiện được bố trí theo 4 vùng rõ ràng:

- RFID RC522 – góc trái trên (SPI: SDA/SCK/MOSI/MISO/RST)
- Servo SG90 – góc phải trên (PWM: GPIO 13)
- LED xanh + LED đỏ + Buzzer – cột phải giữa
- LCD I2C 16x2 – phía dưới giữa (I2C: SDA GPIO 21, SCL GPIO 22)



Hình 3.1: Sơ đồ kết nối đầy đủ các linh kiện trên Wokwi (trạng thái chưa khởi động)

Thiết bị ngoại vi	Chân trên thiết bị	Kết nối đến ESP32	Chức năng
RFID RC522	SDA (SS)	GPIO 5 GPIO 18	Chọn thiết bị SPI
	SCK	GPIO 23 GPIO 19	Xung đồng hồ
	MOSI	GPIO 4 GPIO 13	Dữ liệu chủ → tớ
	MISO	GPIO 26 GPIO 27	Dữ liệu tớ → chủ
	RST	GPIO 25 GPIO 21	Reset module
Servo SG90	Signal (PWM)	GPIO 22	Điều khiển góc quay
LED Xanh	Anode (+)		Báo trạng thái hợp lệ
LED Đỏ	Anode (+)		Báo trạng thái từ chối
Buzzer	Signal		Cảnh báo âm thanh
LCD 16x2 (I2C)	SDA		Dữ liệu I2C
	SCL		Xung đồng hồ I2C

Bảng 4: Bảng kết nối chân giữa các linh kiện và ESP32

3.2.2 Nguyên lý hoạt động

- ① Khởi động: kết nối Wi-Fi, khởi tạo RFID + Servo (đóng 0°) + LED đỏ + LCD hiển thị 'Quét thẻ để mở'
- ② Quét thẻ: RFID RC522 đọc UID từ thẻ/tag đưa đến gần
- ③ Xác thực: so sánh UID với danh sách hợp lệ trong bộ nhớ
- ④a HỢP LỆ → Servo 90°, LED xanh, Buzzer beep, LCD 'CỬA ĐÃ MỞ', Telegram thông báo

- ④b KHÔNG HỢP LỆ → LED đỏ nhấp nháy, Buzzer dài, LCD 'TỪ CHỐI', Telegram cảnh báo
- ⑤ Tự động đóng sau 5 giây → Servo 0°, LED đỏ, Telegram thông báo đóng
- ⑥ Polling Telegram mỗi 2 giây → nhận và thực thi lệnh từ Bot

CHƯƠNG 4: THIẾT KẾ PHẦN MỀM VÀ TRIỂN KHAI

4.1 Nền tảng và công cụ sử dụng

4.1.1 Môi trường mô phỏng phần cứng Wokwi

Dự án sử dụng PlatformIO IDE (extension trong VS Code) kết hợp Wokwi Simulator. Thay vì lập trình trực tiếp trên mạch vật lý trong giai đoạn đầu, hệ thống được thiết kế và thử nghiệm toàn diện trên nền tảng Wokwi. Đây là môi trường mô phỏng điện tử trực tuyến chuyên dụng cho các vi điều khiển như Arduino, ESP32.

- Mô phỏng chính xác: Cung cấp đầy đủ các linh kiện như ESP32, RFID RC522, Servo SG90, LCD, LED, Buzzer với tập lệnh hoạt động sát thực tế.
- Hỗ trợ kết nối mạng: Cung cấp mạng Wi-Fi ảo (SSID: Wokwi-GUEST) cho phép ESP32 truy cập Internet để giao tiếp với Telegram.
- Tính an toàn và tiết kiệm: Kiểm thử logic phức tạp mà không gây rủi ro chập cháy phần cứng.

Cấu trúc thư mục dự án:

```
AutoDoor/  
├── src/main.cpp ← Code chính ESP32  
├── diagram.json ← Sơ đồ phần cứng Wokwi  
├── platformio.ini ← Cấu hình thư viện + board  
└── wokwi.toml ← Liên kết firmware với Wokwi
```

4.1.2 Nền tảng điều khiển Telegram Bot API

Để giải quyết bài toán giao diện người dùng và điều khiển từ xa, đề tài sử dụng Telegram Bot API làm nền tảng giao tiếp trung gian. Thay vì phải lập trình ứng dụng di động riêng biệt, Telegram Bot cung cấp giải pháp giao tiếp hai chiều theo thời gian thực, có tính bảo mật cao và hoạt động đa nền tảng (điện thoại, máy tính bảng, máy tính).

4.1.3 Ngôn ngữ lập trình và Thư viện hỗ trợ

Chương trình điều khiển trung tâm được viết bằng ngôn ngữ C++ dựa trên nền tảng Arduino Core cho ESP32. Các thư viện mã nguồn mở được tích hợp bao gồm:

- MFRC522 (miguelpalboa): Thư viện chuyên dụng để giao tiếp với module RFID RC522 qua SPI.
- ArduinoJson (bblanchon): Xử lý dữ liệu JSON từ Telegram API một cách nhanh chóng và tối ưu bộ nhớ.

- LiquidCrystal_I2C: Điều khiển màn hình LCD 16x2 qua giao tiếp I2C.
- ESP32Servo: Điều khiển servo SG90 bằng tín hiệu PWM trên ESP32.
- WiFi.h và WiFiClientSecure.h: Kết nối Wi-Fi và tạo kết nối HTTPS mã hóa SSL/TLS với Telegram.

4.2 Giải thích code chính (main.cpp)

Code được tổ chức thành các hàm chức năng độc lập, dễ bảo trì và mở rộng:

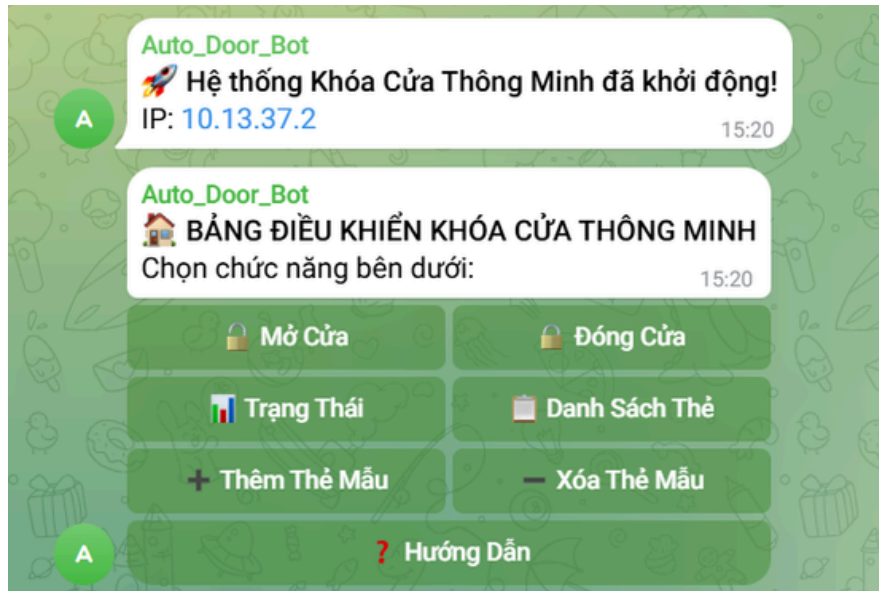
Hàm	Chức năng
getCardUID()	Đọc UID từ RFID, chuyển sang chuỗi hex viết hoa. Duyệt mảng uid.uidByte[] nối vào Stri
isAuthorized(uid)	So sánh UID với mảng authorizedUIDs[10]. Trả về true nếu tìm thấy.
addCard(uid) / removeCard(uid)	Thêm hoặc xóa UID khỏi danh sách hợp lệ (tối đa 10 thẻ).
openDoor(source)	Quay servo 90°, bật LED xanh, beep buzzer, cập nhật LCD, ghi nhận thời gian mở.
closeDoor()	Quay servo 0°, bật LED đỏ, cập nhật LCD, gửi Telegram thông báo đóng cửa.
denyAccess(uid)	Nhấp nháy LED đỏ 3 lần, buzzer dài, hiển thị 'TỪ CHỐI', gửi cảnh báo Telegram.
sendTelegram(message)	HTTP POST đến api.telegram.org/bot{TOKEN}/sendMessage với body JSON.
sendMenuMain()	Gửi tin nhắn có Inline Keyboard 7 nút bố cục 2 cột để điều khiển hệ thống.
handleCommand(cmd, sender, cbId)	Xử lý lệnh text và callback: /open, /close, /status, /listcards, /addcard, /removecard, /help.
processTelegramUpdates()	HTTP GET getUpdates mỗi 2 giây. Phân tích JSON, xử lý message.text và callback_query
loop()	Kiểm tra timeout tự đóng cửa. Polling Telegram mỗi 2 giây. Đọc RFID nếu có thẻ mới.

Bảng 5: Danh sách các hàm chức năng trong main.cpp

4.3 Tích hợp Telegram Bot

4.3.1 Tạo Telegram Bot và cấu hình

- 1. Mở Telegram, tìm @BotFather và nhập lệnh /newbot
- 2. Đặt tên bot và username (phải kết thúc bằng 'bot')
- 3. BotFather cấp BOT_TOKEN dạng 123456789:AAXXXXXX
- 4. Thêm bot vào nhóm chat và cấp quyền Admin
- 5. Lấy CHAT_ID của nhóm (số âm, ví dụ: -5180535191)
- 6. Điền TOKEN và CHAT_ID vào const String trong main.cpp
- 7. Khi hệ thống khởi động, bot tự gửi thông báo và hiện bảng điều khiển



Hình 4.1: Telegram Bot gửi thông báo khởi động và hiển thị menu điều khiển

4.3.2 Điều khiển 2 chiều qua Inline Keyboard

Hệ thống sử dụng Inline Keyboard – các nút bấm nhúng trực tiếp trong tin nhắn. Mỗi nút gửi callback_data về ESP32 qua hàm processTelegramUpdates(), được xử lý qua handleCommand():

Nút Button	Callback Data	Chức năng
🔑 Mở Cửa	/open	Mở khóa cửa từ xa
🔑 Đóng Cửa	/close	Đóng khóa cửa từ xa
Trạng Thái	/status	Xem trạng thái cửa, IP, số thẻ
Danh Sách Thẻ	/listcards	Liệt kê tất cả UID hợp lệ
Thêm Thẻ Mẫu	/addcard_demo	Thêm thẻ mẫu AABBCDD để test
Xóa Thẻ Mẫu	/removecard_demo	Xóa thẻ mẫu AABBCDD
Hướng Dẫn	/help	Hiện lại bảng điều khiển

Bảng 6: Danh sách nút Inline Keyboard và chức năng

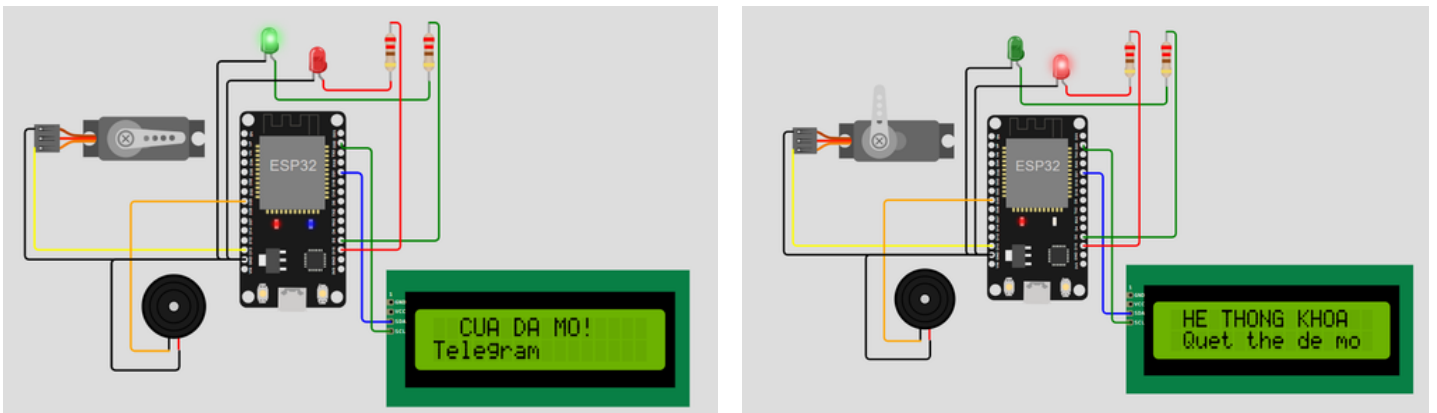
CHƯƠNG 5: KẾT QUẢ THỬ NGHIỆM

5.1 Kết quả mô phỏng

Qua quá trình mô phỏng trên Wokwi, hệ thống đã đạt được các kết quả sau:

- ✓ RFID đọc UID chính xác với các thẻ mô phỏng trên Wokwi
- ✓ Servo phản hồi ngay lập tức ($< 100\text{ms}$) khi nhận lệnh mở/đóng
- ✓ LCD hiển thị đúng thông tin trạng thái trong mọi tình huống
- ✓ LED và Buzzer hoạt động đúng theo logic phân quyền
- ✓ Chức năng tự động đóng cửa sau 5 giây hoạt động ổn định
- ✓ Telegram nhận thông báo thành công qua WiFi Wokwi-GUEST
- ✓ Inline Keyboard hoạt động đúng, bot phản hồi trong 2–4 giây
- ✓ Cảnh báo truy cập bất hợp lệ gửi đúng và kịp thời
- ✓ Thêm/xóa thẻ từ Telegram hoạt động chính xác

Kết quả mô phỏng Wokwi - Trạng thái cửa đóng và mở:



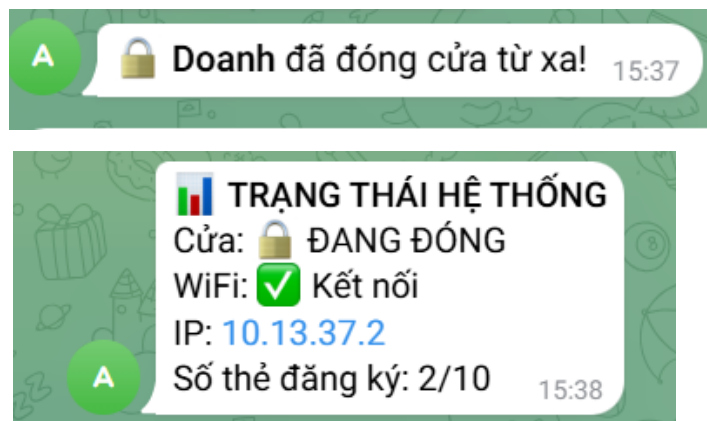
Hình 5.1: Wokwi thông báo cửa đóng và cửa mở

Kết quả kiểm thử mở cửa và tự động đóng sau 5 giây:



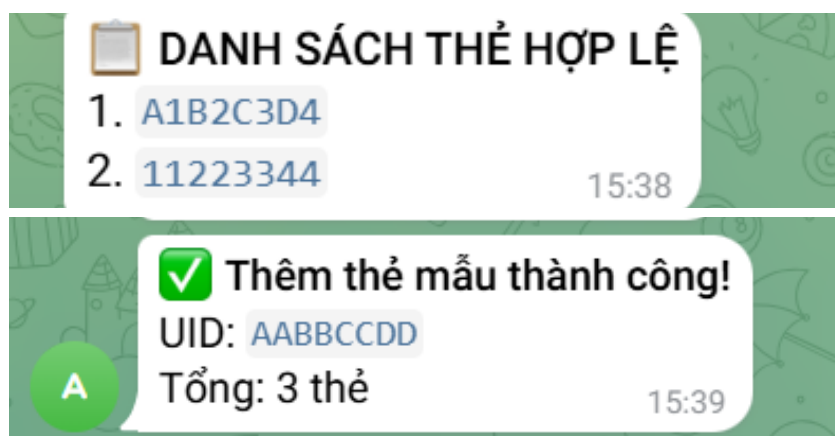
Hình 5.2: Quá trình mở cửa và tự động đóng sau 5 giây

Điều khiển từ xa qua Telegram – Đóng cửa thủ công và xem trạng thái:



Hình 5.3: Người dùng đóng cửa từ xa và xem trạng thái hệ thống

Quản lý danh sách thẻ và thêm thẻ mẫu qua Telegram:



Hình 5.4: Danh sách thẻ hợp lệ và thêm thẻ mẫu

5.2 Hạn chế và hướng phát triển

- Wokwi chưa hỗ trợ đầy đủ RFID thực – cần dùng UID mô phỏng trong môi trường ảo.
- Danh sách thẻ lưu trong RAM, mất khi reset (chưa sử dụng EEPROM để lưu cố định).
- Chưa có tính năng thêm thẻ bằng cách quét thẻ vật lý trực tiếp vào hệ thống.
- Chưa lưu lịch sử truy cập lên cơ sở dữ liệu đám mây như Firebase hay ThingSpeak.
- Chưa tích hợp báo cáo định kỳ và thống kê số lần ra vào theo thời gian.

Hướng phát triển: Tích hợp MQTT broker để quản lý nhiều cửa đồng thời; thêm cảm biến vân tay cho xác thực 2 yếu tố; lưu lịch sử truy cập trên Firebase/ThingSpeak; xây dựng app mobile quản lý thẻ từ xa; tích hợp camera nhận diện khuôn mặt.

CHƯƠNG 6: KẾT LUẬN

Qua quá trình thực hiện tiểu luận, em đã nghiên cứu và hoàn thành một hệ thống khóa cửa thông minh dựa trên ESP32 và RFID với các tính năng nổi bật:

- ✓ Xác thực thẻ RFID an toàn, chính xác, phân quyền rõ ràng (tối đa 10 thẻ)
- ✓ Điều khiển cơ cấu khóa vật lý (servo SG90) theo thời gian thực
- ✓ Thông báo và điều khiển từ xa qua Telegram Bot với Inline Keyboard 7 nút
- ✓ Cảnh báo kịp thời bằng LED, buzzer và Telegram khi có truy cập bất hợp lệ
- ✓ Tự động đóng cửa sau 5 giây, đảm bảo an toàn
- ✓ Mô phỏng đầy đủ trên Wokwi không cần phần cứng thực
- ✓ Code có cấu trúc rõ ràng, dễ bảo trì và mở rộng thêm tính năng

Đề tài đã áp dụng thành công các kiến thức từ môn học Phát triển Ứng dụng IoT: lập trình nhúng với ESP32, giao tiếp ngoại vi (SPI, I2C, PWM), kết nối Wi-Fi, tích hợp API đám mây (Telegram) và mô phỏng với Wokwi. Hệ thống là nền tảng vững chắc để phát triển thành giải pháp bảo mật nhà thông minh hoàn chỉnh với tiềm năng ứng dụng thực tế cao trong các tòa nhà, văn phòng và khu dân cư.

Em xin chân thành cảm ơn giảng viên Võ Việt Dũng đã hướng dẫn tận tình trong suốt quá trình học tập và thực hiện tiểu luận môn học Phát triển Ứng dụng IoT.

TÀI LIỆU THAM KHẢO

- [1] Espressif Systems. (2023). ESP32 Technical Reference Manual. <https://www.espressif.com>
- [2] NXP Semiconductors. MFRC522 Standard performance MIFARE and NTAG frontend – Datasheet. <https://www.nxp.com>
- [3] Telegram. (2024). Telegram Bot API Documentation. <https://core.telegram.org/bots/api>
- [4] Wokwi. (2024). Wokwi IoT & Embedded Systems Simulator. <https://docs.wokwi.com>
- [5] PlatformIO. (2024). PlatformIO IDE for VS Code. <https://platformio.org/platformio-ide>
- [6] Arduino Reference. <https://www.arduino.cc/reference/en/>
- [7] GitHub – miguelbalboa/rfid: MFRC522 Library for Arduino. <https://github.com/miguelbalboa/rfid>
- [8] GitHub – bblanchon/ArduinoJson. <https://arduinojson.org>
- [9] GitHub – madhephaestus/ESP32Servo. <https://github.com/madhephaestus/ESP32Servo>
- [10] GitHub – marcoschwartz/LiquidCrystal_I2C. https://github.com/marcoschwartz/LiquidCrystal_I2C